Task - jenkins

Batch – 7943

Name -Meenakshi

Project - CICD of microservice based application using Jenkins Declarative pipeline over AWS ECS

Prerequisites:

- Setup master slave jenkins setup
- Provide ECS full and ECR permissions to slave node via IAM role
- Install java, awscli, docker on slave node
- Working vote app on ECS

Task:

- Setup a CICD Jenkins declarative pipeline that contains following stages -
    - Git Pull [Checkout SCM]
    - Dockerfile build and Push
    - Deployment over ECS

Optional:

- Add a post build action sending slack messages about the status of the deployment

Answer-:

Prerequisites:

```
ubuntu@ip-172-31-29-251:~$ aws --version
aws-cli/1.36.9 Python/3.12.3 Linux/6.8.0-1016-aws botocore/1.35.68
ubuntu@ip-172-31-29-251:~$ java -version
openjdk version "17.0.13" 2024-10-15
OpenJDK Runtime Environment (build 17.0.13+11-Ubuntu-2ubuntu124.04)
OpenJDK 64-Bit Server VM (build 17.0.13+11-Ubuntu-2ubuntu124.04, mixed mode, sharing)
ubuntu@ip-172-31-29-251:~$ docker -v
Docker version 27.3.1, build ce12230
```

- Setup a CICD Jenkins declarative pipeline that contains following stages -
    - Git Pull [Checkout SCM]

```
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 63e9150ca17af4ed05880d4245e486481f73fcb4 # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git branch -D main # timeout=10
 > git checkout -b main 63e9150ca17af4ed05880d4245e486481f73fcb4 # timeout=10
 > git rev-list --no-walk 63e9150ca17af4ed05880d4245e486481f73fcb4 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Docker Build)
[Pipeline] echo
Building Docker image...
[Pipeline] sh
+ docker build -t 230231799530.dkr.ecr.us-east-1.amazonaws.com/service/voting:votin_app ./vote
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 1.08kB done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 DONE 0.1s

#3 [internal] load .dockerignore
```

```
Started by user meenakshi
[Pipeline] Start of Pipeline
[Pipeline] node
Running on slave_machine in /home/ubuntu/workspace/workspace/task
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Pull)
[Pipeline] echo
Pulling code from GitHub repository...
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Fetching changes from the remote Git repository
Checking out Revision 63e9150ca17af4ed05880d4245e486481f73fcb4 (refs/remotes/origin/main)
Commit message: "Merge pull request #380 from dockersamples/change-ports"
 > git rev-parse --resolve-git-dir /home/ubuntu/workspace/workspace/task/.git # timeout=10
 > git config remote.origin.url https://github.com/dockersamples/example-voting-app.git # timeout=10
Fetching upstream changes from https://github.com/dockersamples/example-voting-app.git
 > git --version # timeout=10
 > git --version # 'git version 2.43.0'
 > git fetch --tags --force --progress -- https://github.com/dockersamples/example-voting-app.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
```

- Dockerfile build and Push

```
#4 [base 1/5] FROM docker.io/library/python:3.11-
slim@sha256:e8381c802593deb0c4d25bd3f4e05e94382f6bf33090de22679fc7488cd68bbb
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 274B done
#5 DONE 0.0s

#6 [base 4/5] COPY requirements.txt ./requirements.txt
#6 CACHED

#7 [base 2/5] RUN apt-get update &&     apt-get install -y --no-install-recommends curl &&     rm -rf
/var/lib/apt/lists/*
#7 CACHED

#8 [base 3/5] WORKDIR /usr/local/app
#8 CACHED

#9 [base 5/5] RUN pip install --no-cache-dir -r requirements.txt
#9 CACHED

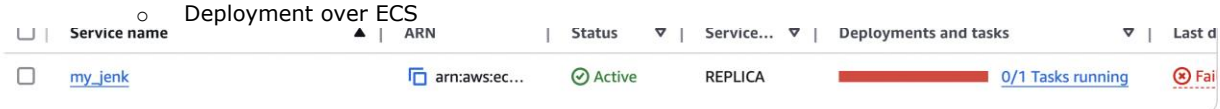
#10 [final 1/1] COPY . .
#10 CACHED
```

```
[Pipeline] stage
[Pipeline] { (Docker Push to ECR)
[Pipeline] echo
Pushing Docker image to Amazon ECR...
[Pipeline] sh
+ aws ecr get-login-password --region us-east-1
+ docker login --username AWS --password-stdin 230231799530.dkr.ecr.us-east-1.amazonaws.com/service/voting
WARNING! Your password will be stored unencrypted in /home/ubuntu/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

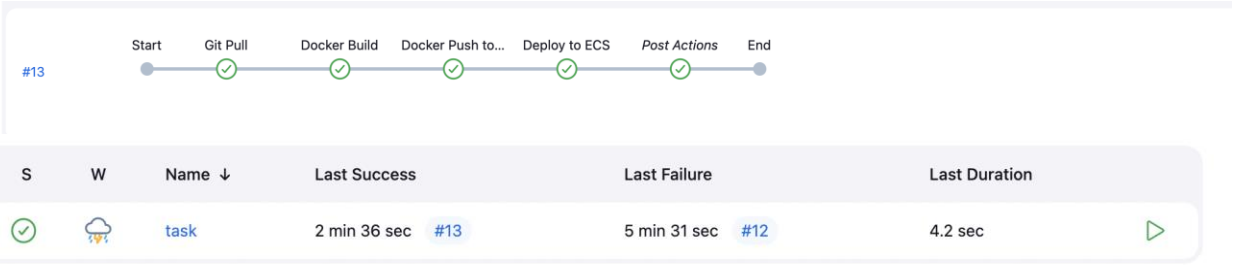
Login Succeeded
+ docker push 230231799530.dkr.ecr.us-east-1.amazonaws.com/service/voting:votin_app
The push refers to repository [230231799530.dkr.ecr.us-east-1.amazonaws.com/service/voting]
7cd071f4c42d: Preparing
a5b32978f3e2: Preparing
f258ff1d62e6: Preparing
4e665ed53c1f: Preparing
060b5b7f6b10: Preparing
bb4412c1f640: Preparing
25b72b8029f8: Preparing
2440dc0bfce0: Preparing
c3548211b826: Preparing
```



- Deployment over ECS



```
        "enableECSManagedTags": true,
        "propagateTags": "NONE",
        "enableExecuteCommand": false,
        "availabilityZoneRebalancing": "ENABLED"
    }
}
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline execution completed!
[Pipeline] echo (hide)
Application deployed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☁	task	2 min 36 sec #13	5 min 31 sec #12	4.2 sec

Agent where it ran

	slave_machine	Linux (amd64)	In sync	4.67 GiB	! 0 B	4.67 GiB	71ms
	Data obtained	**7 min 52 sec**	**7 min 52 sec**	**7 min 52 sec**	**7 min 52 sec**	**7 min 52 sec**	**7 min 52 sec**

Cloned on this directory

```
ubuntu@ip-172-31-29-251:~/workspace/workspace/task$ ls
LICENSE      README.md                  docker-compose.images.yml  docker-stack.yml  k8s-specifications  seed-data  worker
MAINTAINERS  architecture.excalidraw.png  docker-compose.yml         healthchecks      result              vote
```

Pipeline content

```
pipeline {
  agent {
    label 'docker-enabled-slave' // Label matching the slave with Docker installed
  }
  environment {
```

```
    // Replace these placeholders with actual values specific to your setup
    ECR_REPO_URI = '230231799530.dkr.ecr.us-east-1.amazonaws.com/service/voting' // ECR repository URI
    AWS_REGION = 'us-east-1' // AWS region
    IMAGE_NAME = 'example-voting-app' // Docker image name
    IMAGE_TAG = 'votin_app' // Image tag
    CLUSTER_NAME = 'service' // ECS cluster name
    SERVICE_NAME = 'my_jenk' // ECS service name
  }
  stages {
    stage('Git Pull') {
      steps {
        echo 'Pulling code from GitHub repository...'
        git branch: 'main' ,url: 'https://github.com/dockersamples/example-voting-app.git'
      }
    }
    stage('Docker Build') {
      steps {
        echo 'Building Docker image...'
        sh """
          docker build -t ${ECR_REPO_URI}:${IMAGE_TAG} ./vote
        """
      }
    }
    stage('Docker Push to ECR') {
      steps {
```

```
            echo 'Pushing Docker image to Amazon ECR...'
            sh """
                aws ecr get-login-password --region ${AWS_REGION} | docker login --username
AWS --password-stdin ${ECR_REPO_URI}
                docker push ${ECR_REPO_URI}:${IMAGE_TAG}
            """
        }
    }
    stage('Deploy to ECS') {
        steps {
            echo 'Deploying the service on ECS...'
            sh """
                aws ecs update-service \
                    --cluster ${CLUSTER_NAME} \
                    --service ${SERVICE_NAME} \
                    --force-new-deployment \
                    --region ${AWS_REGION}
            """
        }
    }
}
post {
    always {
        echo 'Pipeline execution completed!'
    }
    success {
```

```
        echo 'Application deployed successfully.'
    }
    failure {
        echo 'Pipeline failed. Please check the logs.'
    }
  }
}
```

Script ?

```
agent {
    label 'docker-enabled-slave' // Label matching the slave with Docker installed
}
environment {
    // Replace these placeholders with actual values specific to your setup
    ECR_REPO_URI = '230231799530.dkr.ecr.us-east-1.amazonaws.com/service/voting' // ECR repository URI
    AWS_REGION = 'us-east-1' // AWS region
    IMAGE_NAME = 'example-voting-app' // Docker image name
    IMAGE_TAG = 'votin_app' // Image tag
    CLUSTER_NAME = 'service' // ECS cluster name
    SERVICE_NAME = 'my_jenk' // ECS service name
}
stages {
    stage('Git Pull') {
        steps {
            echo 'Pulling code from GitHub repository...'
            git branch: 'main' ,url: 'https://github.com/dockersamples/example-voting-app.git'
        }
    }
    stage('Docker Build') {
        steps {
            echo 'Building Docker image...'
            sh """
                docker build -t ${ECR_REPO_URI}:${IMAGE_TAG} ./vote
            """
        }
    }
    stage('Docker Push to ECR') {
        steps {
            echo 'Pushing Docker image to Amazon ECR...'
            sh """
                aws ecr get-login-password --region ${AWS_REGION} | docker login --username AWS --password-stdin ${ECR_REPO_URI}
                docker push ${ECR_REPO_URI}:${IMAGE_TAG}
            """
        }
    }
    stage('Deploy to ECS') {
        steps {
            echo 'Deploying the service on ECS...'
            sh """
                aws ecs update-service \
                    --cluster ${CLUSTER_NAME} \
                    --service ${SERVICE_NAME} \
                    --force-new-deployment \
                    --region ${AWS_REGION}
            """
        }
    }
}
post {
    always {
        echo 'Pipeline execution completed!'
    }
    success {
        echo 'Application deployed successfully.'
    }
    failure {
        echo 'Pipeline failed. Please check the logs.'
    }
}
}
```